



SH320 Data Engineering

Spring 2026 Semester

6 Data Acquisition

Hyun Ahn

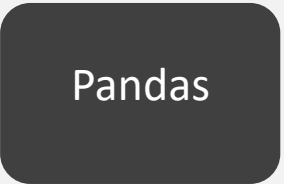
School of Computing and Artificial Intelligence

hyunahn@hs.ac.kr

Core Libraries for Data Science

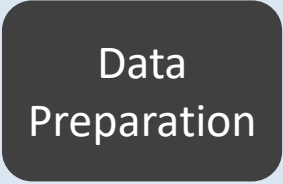
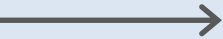
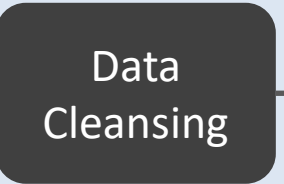
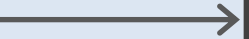
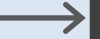
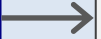


- **[W2] N-Dimensional Arrays**
 - NumPy arrays
 - Array manipulation
- **[W3] Efficient Computation in NumPy**
 - Universal functions
 - Advanced features



- **[W5] Data Manipulation: Part 1**
 - Pandas objects: Series, DataFrame
 - Implicit / explicit indexing
- **[W6] Data Manipulation: Part 2**
 - Pandas objects & manipulation
 - Combining datasets
 - Exploratory data analysis

Data Engineering



- **[W7] Data Acquisition**
 - Web scraping
 - RESTful API

Data Acquisition Methods



- 데이터 취득 방법

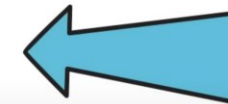
- Private data: 데이터베이스, 시스템 로그 등
- 공개 데이터셋 저장소 및 포털
 - AI: Kaggle, Google Dataset Search, UCI Machine Learning Repository, AI허브
 - 통계: 공공데이터포털, 서울 열린 데이터 광장, Data.gov
- 크롤링 / 웹스크래핑
- API 활용
 - 예(기상): OpenWeatherMap API
 - 예(경제): FRED API
 - 예(지도): Kakao Map API
 - 예(뉴스): News API



②

DATA ACQUISITION

- WEB SERVERS
- LOGS
- DATABASES
- API'S
- ONLINE REPOSITORIES

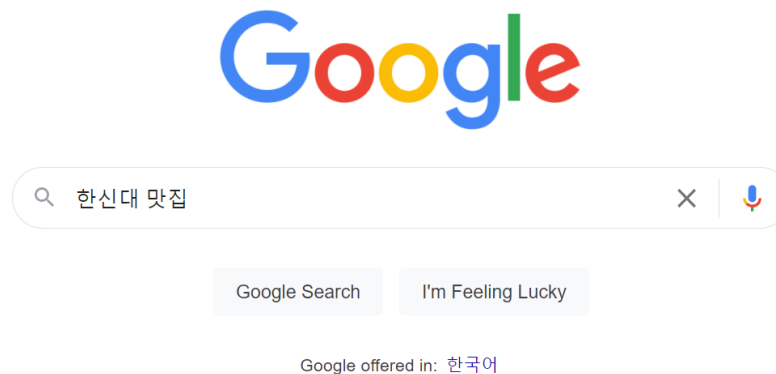


6.1 웹 스크래핑



수렵 채집: Hunting and Gathering

현대인 = Information Hunter-Gatherers



우대사항

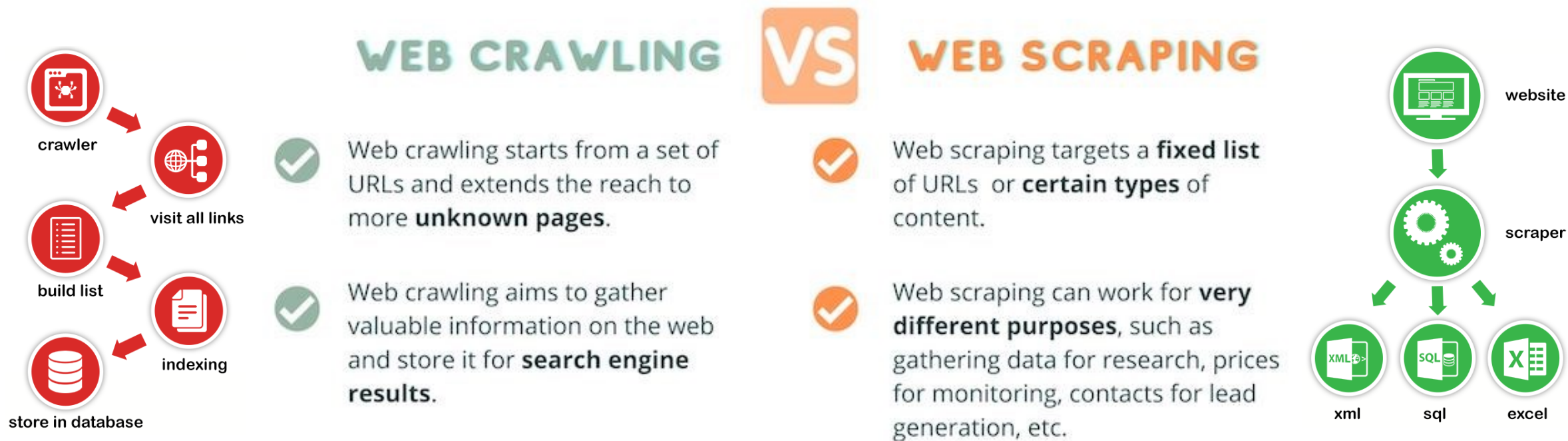
- 스포츠 통계 및 머신러닝 분야의 영문 논문을 읽고 이해하는 능력
- 시중 인기 게임(League of Legends, Valorant, PUBG 등)에 대한 이해도
- Tensorflow, Pytorch를 이용한 딥러닝 프로젝트 경험에 있으신 분
- MySQL를 이용한 데이터 추출 및 전처리 역량
- 야구, 농구 등 전통 스포츠의 데이터 분석 방법론에 대한 이해도
- 개발 블로그 및 GitHub를 꾸준히 관리하시는 분
- 수학, 통계학, 데이터 과학, 컴퓨터 과학 등 관련 분야 박사 학위

What is Web Scraping?

- 웹 스크래핑: 자동화된 SW(= bot)를 사용하여 웹 사이트의 콘텐츠와 데이터를 추출하는 것
 - 자동 웹 네비게이션을 통한 내용 추출
 - HTTP 프로토콜을 이용한 특정 웹페이지 내용 추출



Web Crawling vs. Web Scraping



일반적으로 웹 크롤링은 검색 엔진(예: Google, Bing)이 인터넷에 연결된 웹 사이트들을 방문하면서 대규모로 웹 페이지의 구조 및 링크들을 탐색하고 웹 콘텐츠를 인덱싱하는 기술을 의미합니다.

반면에, 웹 스크래핑은 특정 웹 페이지로부터 데이터 분석 및 연구 등의 목적을 위해 필요한 데이터 만을 추출하는 기술을 말합니다. 🐼

The Web Scraping Process



- **일반적인 웹 스크래핑 절차**

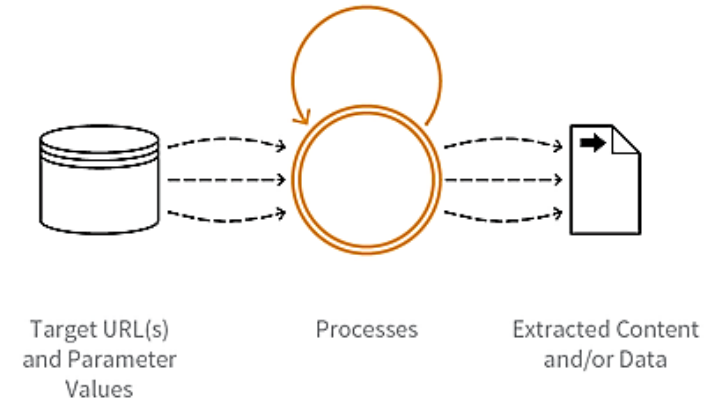
1. 대상 웹 사이트 식별
2. 데이터 추출하고자 하는 웹페이지들의 URL 수집
3. URL에 HTML 문서 요청
4. 반환된 HTML 문서를 검사 및 원하는 데이터 추출
5. JSON 또는 .csv 파일 또는 기타 구조화된 형식으로 데이터를 저장

Is Web Scraping Legal?



- 웹 스크래핑의 사용 목적

- 시장 조사(예: 가격 비교)
- 빅데이터/AI 를 위한 학습용 데이터 수집
- 비즈니스 관련 데이터베이스 구축
- pdf 출판물로부터 정보를 추출(예: Google Scholar)



Source: OWASP ([link](#))

- **위법 이슈:** 기본적으로 웹 스크래핑은 웹사이트의 콘텐츠를 복제하는 행위

- OWASP¹에서는 웹 스크래핑을 자동화된 위협 요소로 정의(OAT-011)

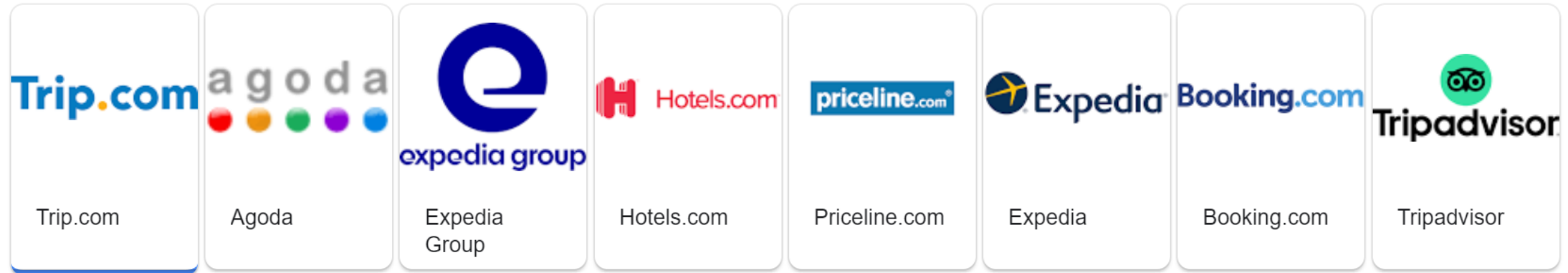
What are Valid Uses for Web Scraping?



- 합법적 사용 예:

- 검색 엔진 크롤러^{crawlers}: Google, Bing
- 온라인 포럼 및 소셜 미디어 데이터 수집(예: 시장 조사 목적)
- 온라인 소매업체에서 가격 및 제품 설명 수집(예: 가격 비교 웹사이트)

Related to Trip.com and Tripadvisor



Malicious Use Cases for Web Scrapping



- **불법적 사용:** 소유자 허락하지 않은 데이터를 무단으로 추출한 경우
 - 예: **사람인**이 경쟁사인 **잡코리아**의 채용정보를 추출하고 상업적으로 활용
 - 판결: 웹 스크래핑을 통해 확보한 콘텐츠를 자신의 영업에 무단 사용하는 것은 불법이다.
 - 예: **HiQ Labs**가 **LinkedIn**의 프로필 정보를 수집하고, 이를 다른 정보들과 함께 가공/분석하여 판매
 - 판결: 로그인 없이 필요한 정보는 공공재이므로, 불법 아니다.

saramin vs JOBKOREA

hiQ vs LinkedIn

Libraries Used in Python



- **Requests:** Python 에서 제공하는 **HTTP Request 라이브러리**
 - `pip install requests`
- **Beautiful Soup:** Requests 로부터 얻어온 HTML 내용에서 필요한 데이터를 추출하기 위한 **파싱 라이브러리**
 - `pip install beautifulsoup4`
 - 단점: 동적 웹페이지로부터 데이터를 추출하기 어려움
- **Selenium:** 동적 페이지 데이터 추출에 유용한 파싱 라이브러리
 - 웹드라이버를 통해 브라우저를 제어

Chrome Inspector

• Chrome Inspector 기능을 이용한 HTML 요소 찾기

– F12 ⇨ Ctrl + Shift + C



Exercise: Search News by “메타버스”



- 예제: 메타버스 키워드로 뉴스 검색하기
 - Requests 를 이용하여 url 에 해당되는 HTML 내용 받기

```
import requests

url =
'https://search.naver.com/search.naver
?where=nexearch&sm=top_h ty&fbm=0&ie=utf
8&query=%EB%A9%94%ED%83%80%EB%B2%84%E
C%8A%A4'

res = requests.get(url)

html_doc = res.text
```



```
<!doctype html> <html lang="ko">
<head> <meta charset="utf-8"> <meta
name="referrer" content="always">
<meta name="format-detection"
content="telephone=no,address=no,em
ail=no"> <meta name="viewport"
content="width=device-
width,initial-scale=1.0,maximum-
scale=2.0"> <meta
property="og:title" content="메타버
스 : 네이버 통합검색" />
```


Exercise: Search News by “메타버스”



- BeautifulSoup 파서 생성 및 HTML 문서 구조화

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_doc, 'html.parser')
print(soup.prettify())
```



```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="utf-8" />
    <meta content="always" name="referrer" />
    <meta content="telephone=no,address=no,email=no"
name="format-detection" />
    <meta content="width=device-width,initial-scale=1.0,maximum-
scale=2.0" name="viewport" />
    <meta content="메타버스 : 네이버 통합검색" property="og:title">
```

Exercise: Search News by “메타버스”



- `find_all()`: 조건에 맞는 태그 정보를 검색

– `<a>` 태그 검색

```
soup.find_all('a')
```



```
[<a href="#lnb"><span>메뉴 영역으로 바로가기</span></a>,  
  <a href="#content"><span>본문 영역으로 바로가기  
</span></a>,  
  <a class="link" href="https://www.naver.com"  
onclick="return goOtherCR(this,  
'a=sta.naver&r=&i=&u='+urlencode(this.hr  
ef));"><i class="spnew ico_logo">NAVER</i></a>,  
  <a aria-pressed="false" class="bt_setkr" href="#"  
id="ke_kbd_btn" onclick="return tCR('a=sch.ime');"  
role="button"><i class="spnew ico_keyboard">한글 입력기  
</i></a>
```

Exercise: Search News by “메타버스”



- 메서드 시그니처: `find_all(name, attrs, recursive, string, limit, **kwargs)`
- 뉴스 정보 추출: ‘`class`’ 속성이 ‘`news_tit`’ 인 `<a>` 태그 검색

```
news = soup.find_all("a", {"class": "news_tit"})
```



```
[<a class="news_tit"
href="https://www.yna.co.kr/view/AKR20231030020100371?input=1195m"
onclick="return goOtherCR(this,
'a=nws_all*e.tit&r=1&i=880000D8_000000000000000000014295502&g=001.001429
5502&u='+urlencode(this.href));" target="_blank" title='"일본 교과서 왜곡 막자"...반
크, 메타버스 독도 전시관 구축'">"일본 교과서 왜곡 막자"...반크, <mark>메타버스</mark> 독도 전시관 구
축</a>, (생략)]
```

Exercise: Search News by “메타버스”



– 뉴스 제목만 추출

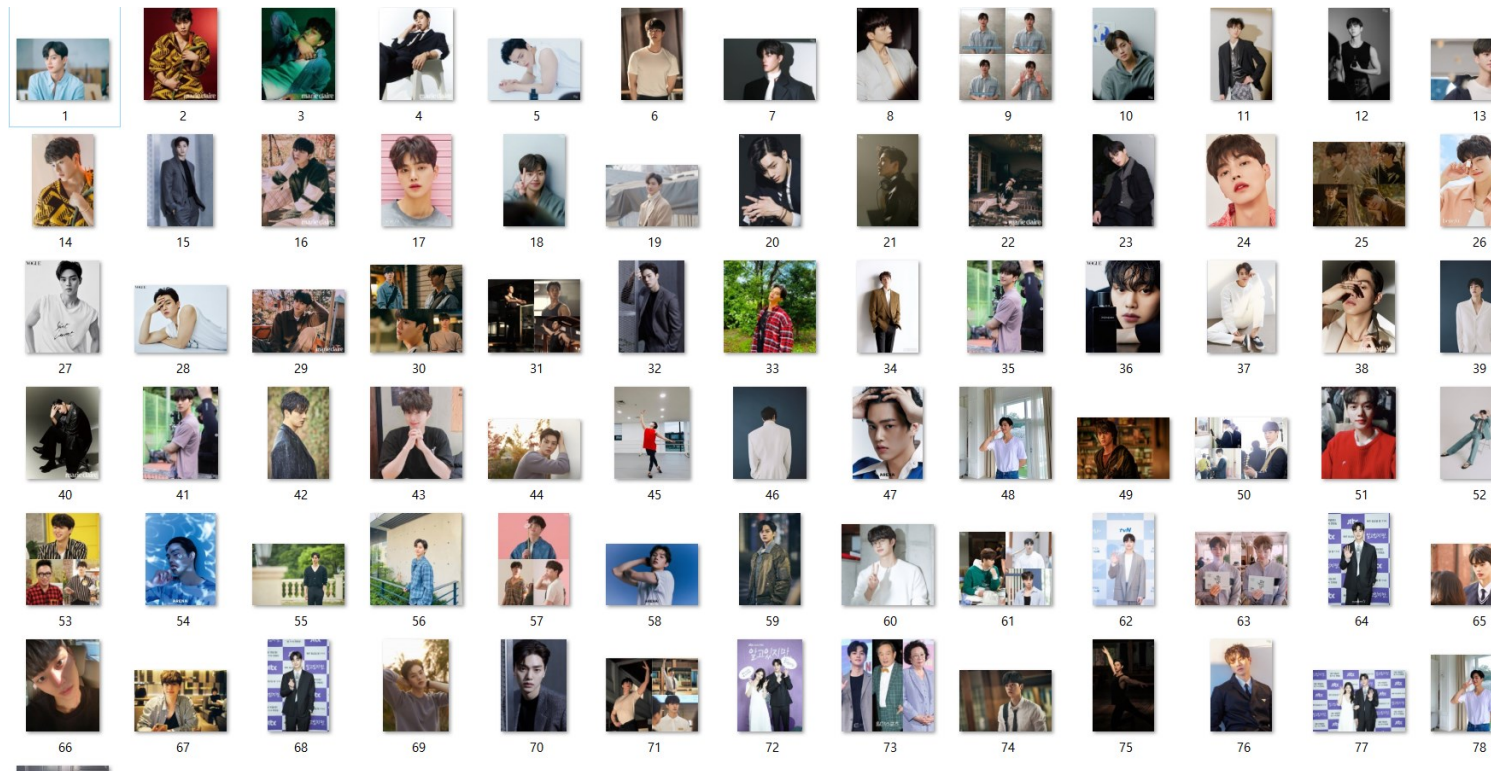
```
for i in range(len(news)):
    print(news[i].get('title'))
```



"일본 교과서 왜곡 막자"...반크, 메타버스 독도 전시관 구축
메타버스 기업 성과는...11월7일 '엔알피×넥시드 데모데이' 개최
배재대, 청주 양청고서 '청소년 행복 메타버스 스쿨' 개최
대전 서구 가수원도서관 '메타버스 여행하기' 교육 운영

Exercise: Image Scraping

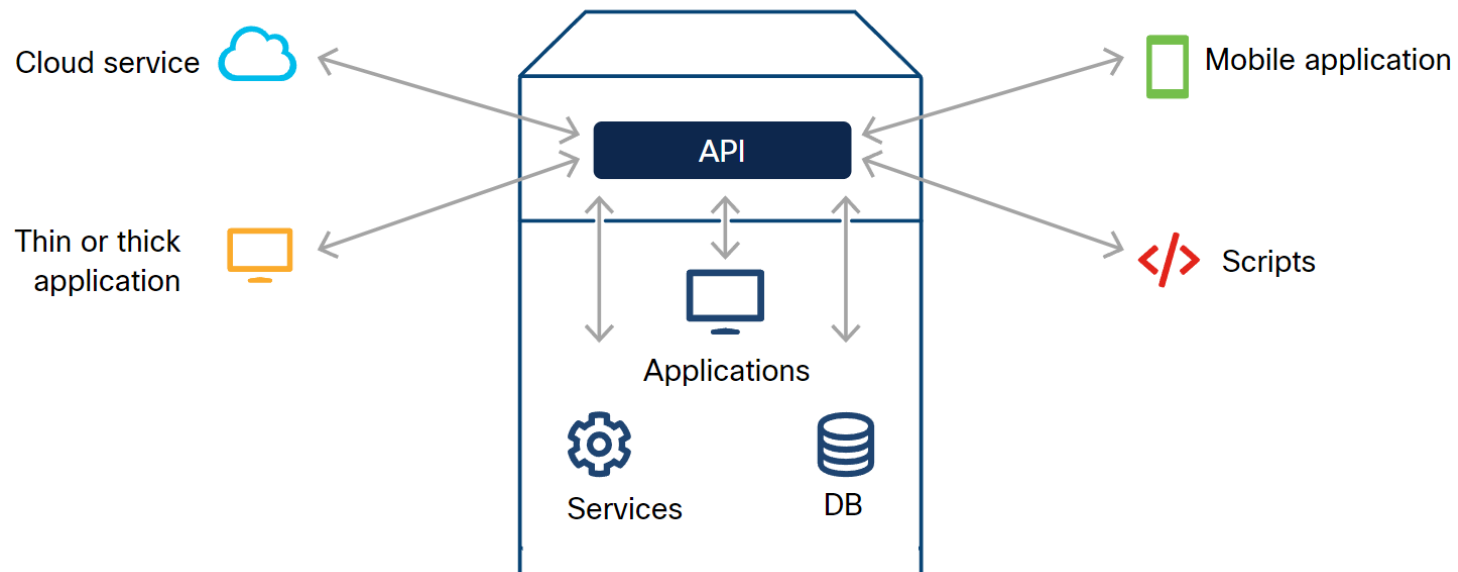
- 예제: 이미지 스크래핑
 - 필요 라이브러리: Requests, BeautifulSoup, dload, Selenium
 - 출처: 송강 이미지 웹 스크래핑 ([link](#))



6.2 RESTful API

What is an API?

- **API** Application Programming Interface : 애플리케이션이나 디바이스가 서로 간에 통신할 수 있는 방법을 정의하는 규칙
 - 일반적으로 웹 기반 통신 프로토콜을 사용 = **웹서비스** Web Services
 - API를 통해 외부에 제공되는^{exposed} 데이터, 서비스, 기능 등이 결정됨



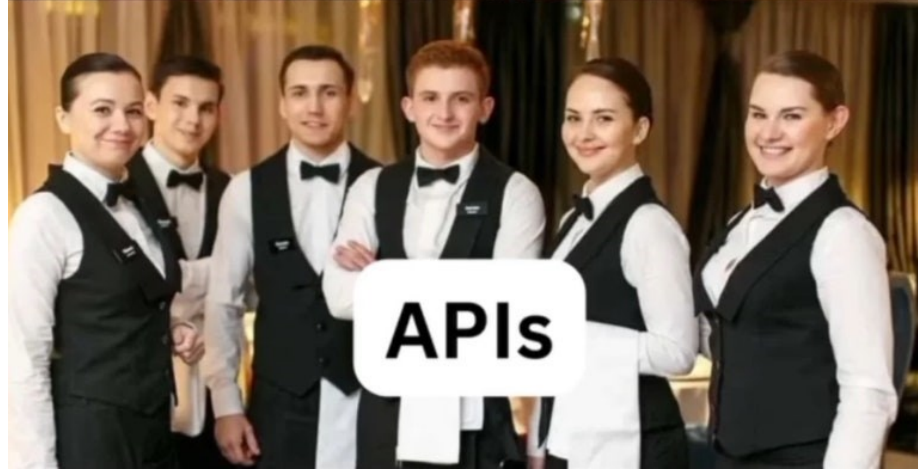
Backend



Frontend



APIs



Why Use APIs?



- API는 외부 애플리케이션에서 접근 가능하며, 다양한 목적으로 사용된다.
 - 기능 제공 : API에서 제공하는 기능을 내가 개발하는 애플리케이션에 포함
 - 데이터 취득 : 애플리케이션에 필요한 데이터들을 API를 통해 획득
 - 태스크 자동화 : 수동으로 처리해야 하는 작업들을 API 호출을 통해 자동화
 - 예: 인보이스 처리 워크플로우
 - 예: Agentic AI 기반의 서비스

😊 Agentic AI의 가장 큰 특징은 사용자 대신 행동(Action)한다는 것입니다.

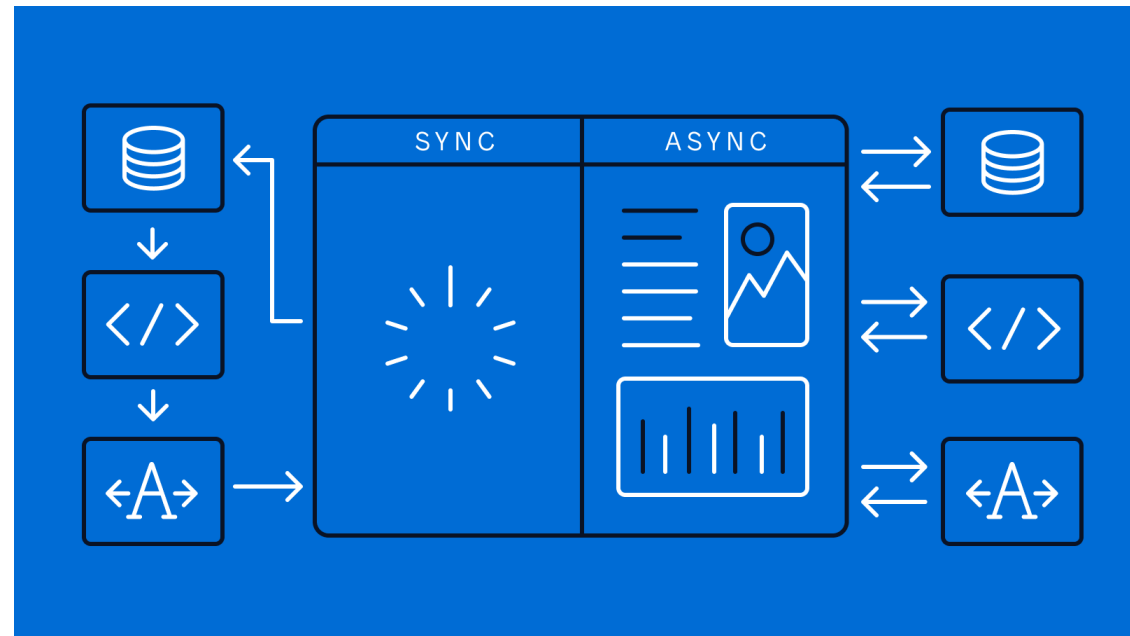
하지만 소프트웨어인 AI가 현실 세계나 다른 디지털 환경에 개입하려면 반드시 통로가 필요하며, 그 통로가 바로 API입니다.

- 예: "다음 주 금요일 제주도 출장 일정을 잡고 비행기와 숙소를 예약해 줘"

⇒ AI는 항공사 API와 호텔 예약 API를 호출하여 실제로 결제와 예약을 진행

Types of Design Styles

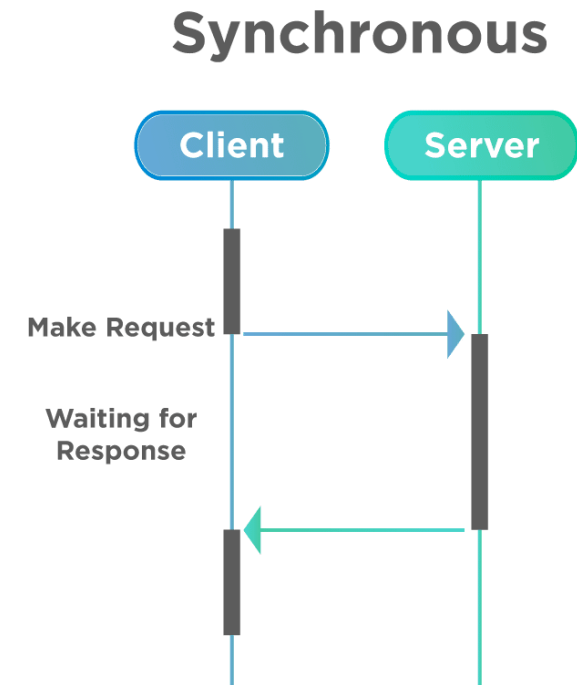
- 동기식 Synchronous 와 비동기식 Asynchronous API 설계
 - API 설계할 때 고려해야 하는 주요 이슈
 - 애플리케이션은 API의 설계 유형(동기식 | 비동기식)에 따라 다르게 구현됨



Synchronous APIs



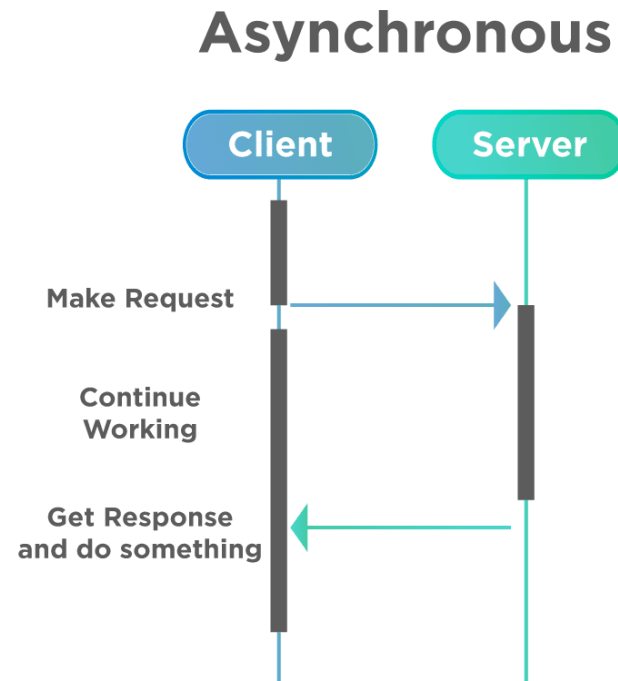
- 동기식 API는 클라이언트로부터 요청을 받은 즉시 응답을 시도한다.
- 동기식 API 설계의 장점
 - 요청 받은 데이터가 가용성이 높고 빠른 응답이 가능할 경우, 높은 애플리케이션 성능을 기대할 수 있음
 - 구현이 쉬움
- 클라이언트 측 처리
 - 애플리케이션은 응답을 받을 때까지 대기
 - 응답을 받아야만 다음 후속 태스크를 실행할 수 있음



Asynchronous APIs



- 비동기식 API는 요청과 응답을 분리된 태스크로 비동기식으로 처리한다.
- 비동기식 API 설계의 장점
 - 애플리케이션이 응답을 기다리는 동안 다른 작업을 수행 할 수 있음
 - 응답 데이터를 생성하는 데 시간이 걸리는 경우에 적합 (대부분의 웹 애플리케이션에 해당)
 - 비동기식 처리 예:
 - 댓글 달기 / 삭제 (새로고침 없이)
 - 무한 스크롤
 - 실시간 검색 자동완성
 - 채팅창
 - 대시보드 실시간 업데이트



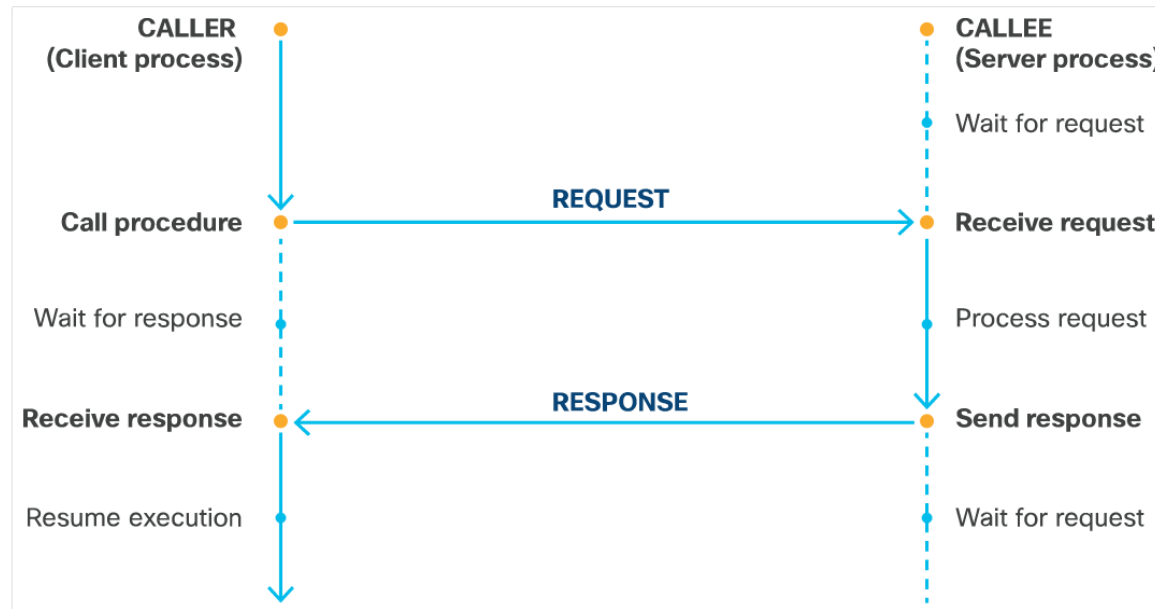
Common Protocols and Architectural Styles



- API 설계와 관련된 다양한 표준, 프로토콜, 설계 형식이 존재함
- 대표적인 API 아키텍처 스타일
 - **RPC** Remote Procedure Call
 - **SOAP** Simple Object Access Protocol
 - **REST** Representational State Transfer

Remote Procedure Call (RPC)

- 전통적인 요청-응답 Request-Response 모델에 기반한 API 프로토콜
 - 클라이언트가 서버에 있는 프로시저(함수)를 로컬 시스템에서 호출하듯이 사용
 - 클라이언트는 필요한 매개변수를 가지고 서버의 함수를 호출하고 서버는 그 함수를 실행한 뒤 결과를 클라이언트에게 전송



Simple Object Access Protocol (SOAP)



- XML 기반 메시징 프로토콜

- 이기종^{Heterogeneous} 플랫폼 및 언어로 작성된 애플리케이션 사이의 통신을 지원
(= 플랫폼&언어 독립적)
- 프로토콜 중립성 : 주요 통신 프로토콜과 잘 호환됨 (예: HTTP, SMTP, TCP, UDP)
- 엄격한 메시지 처리(예: 에러 처리, 보안)
- 모든 내용들은 SOAP Envelope 내에 정의됨

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring>Query request too large.</faultstring>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```


Representational State Transfer (REST)

- HTTP 프로토콜 기반의 API 아키텍처

- SOAP과 비교하여 **단순하고 구축이 용이**해서, 최근의 Open API들은 대부분 REST 기반의 API로 구축됨



- REST API 요청의 구성 요소

- URI Uniform Resource Identifier
- HTTP Method (GET, POST, PUT, DELETE 등)
- Header
- Body

Standard HTTP Methods



- RESTful API는 표준 HTTP 메서드들을 이용
 - 무엇을 어떻게 할 것인가?
URI HTTP
 메서드

표. HTTP 메서드와 CRUD 액션 매핑

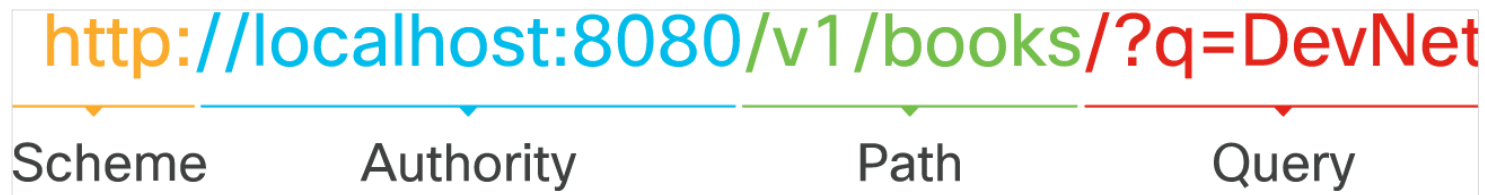
HTTP Method	CRUD Action	Description
POST	Create	Create a new object or resource.
GET	Read	Retrieve resource details from the system.
PUT	Update	Replace or update an existing resource.
PATCH	Partial Update	Update some details from an existing resource.
DELETE	Delete	Remove a resource from the system.

Representational State Transfer (REST)



- **REST API의 URI**

- Scheme: 프로토콜을 정의
- Authority: 호스트와 포트 번호를 정의
- Path: 리소스(데이터, 객체, 서비스 등)의 경로
- Query: API 호출에 필요한 추가 정보 (예: 매개변수)



- 참고: KAKAO Developers, REST API ([link](#))

Exercise: CoinGecko



- 예제: **CoinGecko API를 이용한 알트코인 시가총액 시각화**
 - CoinGecko: DeFi 코인의 순위 및 시가총액 데이터를 실시간으로 제공하는 플랫폼 ([link](#))
 - CoinGecko API Documentation ([link](#))
 - Endpoint Overview: API 엔드포인트 (URL) 정의

References



- Panda, S., [A beginner's guide to web scrapping using BeautifulSoup](#), Analytics Vidhya ([link](#))
- D. Bevans, [Asynchronous and synchronous programming: What's the difference?](#), mendix ([link](#))
- [find_all\(\) method](#), BeautifulSoup Documentation ([link](#))
- Arora, L., [Hands-On Introduction to Web Scraping in Python: A Powerful Way to Extract Data for your Data Science Project](#), Analytics Vidhya ([link](#))
- 유병혁, [Python: Requests와 BeautifulSoup를 이용한 파싱\(parsing\)](#), 다음블로그 ([link](#))
- Lazar, D., [Scraping Medium with Python & BeautifulSoup](#), Medium ([link](#))
- [What Is a Web Crawler and How Does It Work](#), Octoparse ([link](#))
- Foo, E., [What is Web Scraping and How Does It Work](#), Data Science Central ([link](#))
- Tovar, L. J., [Web Scraping: What is it and what is it used for?](#), Medium ([link](#))



수고하셨습니다.